

Design Note — LLD Practice Platform

1. MVP architecture

The prototype has two layers:

- React frontend for problem browsing, submission, feedback and history.
- Express backend for problem data, submissions and evaluation.

The backend is the source of truth for evaluation.

React UI

|

| HTTP/JSON

v

Express API

|

+---- ProblemService

|

+---- SubmissionService

|

+---- Evaluator

|

+---- AttemptStore

2. User flow

Problems

↓

Open problem

↓

Write solution

↓

Submit

↓

Deterministic evaluation

↓

Feedback

↓

History

↓

Retry

3. Domain concepts

Problem

Represents an LLD exercise with a title, statement, requirements and evaluation hints.

Submission

Represents one learner attempt.

Evaluation

Contains score, strengths, issues and suggestions.

Attempt

A submission plus its evaluation and timestamp.

4. Core LLD examples inside the problem content

Parking Lot uses:

- `Vehicle`
- `ParkingSpot`
- `ParkingFloor`
- `ParkingLot`
- `SpotAllocationStrategy`

The strategy interface keeps allocation policy replaceable.

Vending Machine encourages:

- explicit states

- encapsulation
- state transitions

Elevator encourages:

- interfaces
- responsibility separation
- scheduling strategy

5. Evaluation

The baseline evaluator checks signals in submitted text:

- interface/abstraction
- multiple classes
- separation of responsibilities
- relevant design pattern
- domain relationships

Each signal contributes two points, capped at ten.

Validation rejects an empty or very short solution.

6. Failure and edge cases

- Unknown problem ID → 404.
- Empty submission → 400.
- Unknown submission ID → 404.
- Very short solution → 400.
- Evaluation always returns a structured response.
- Attempts are kept separately, so retry creates a new history item.

7. Trade-offs

In-memory storage

Fast to implement and sufficient for a demo, but data disappears after restart.

Rule-based evaluation

Predictable and testable, but less nuanced than expert review.

Single service

Simple to run and understand, but not independently scalable.

No authentication

Keeps the MVP focused. Production would need user accounts and authorization.

8. Future improvements

- PostgreSQL/MongoDB persistence.
- Authentication.
- Code/diagram upload.
- AST-based code analysis.
- LLM-assisted explanation.
- Mentor review.
- More problems and rubrics.
- Analytics showing improvement across attempts.